SOMAIYA VIDYAVIHAR UNIVERSITY

K. J. Somaiya School of Engineering
Mumbai 400077

LAB CA IMPLEMENTATION REPORT

ML Based Predictive Cloud Resource Management

SLA aware auto scaling cost optimization and adaptive workload handling

Advanced Cloud Computing
Computer Engineering | LY B.Tech | Semester VII
Academic Year 2026-2027
25 October 2026

Implementation platform: Local Kubernetes
Docker Desktop and Minikube
Machine learning approach: Ridge regression

Submitted by

Parth Dhande (16010123232)
Purab Madhan (16010123242)
Pushkar Rai (16010123243)

Abstract

We implemented an ML based resource manager that forecasts near future HTTP demand and changes the number of application workers on a real local Kubernetes cluster. A Ridge regression model predicts the peak request rate over the next 30 seconds using the preceding 60 seconds of demand. The controller combines that forecast with observed demand, latency and error safeguards, replica limits and conservative downscaling.

Twelve matched runs compared fixed, reactive and guarded predictive policies on two workloads and two seeds. Guarded scaling had no violating scored windows in this suite and used 10.7% fewer replica seconds than fixed capacity on periodic traffic. It required more capacity than reactive scaling. The model outperformed persistence on a synthetic holdout but underperformed it across 64 aligned replay targets. Deliberately incorrect forecasts show why independent service safeguards are necessary. Latency, errors, CPU use and replica changes are measured; financial cost and energy are analytical estimates.

Contents

1 Introduction and system architecture 2
2 Implementation platform and input data 3
3 Machine learning methodology 4
4 Calibration and resource allocation 5
5 Safeguards and implemented functionality 6
6 Execution demonstration 7
7 Experimental method and performance results 9
8 Utilization cost and energy 10
9 Prediction failure and recovery 11
10 Limitations and conclusion 13
11 Reproduction and references 14

The report documents completed execution. The code repository and the accompanying three person presentation are identified in Section 11.

1 Introduction and system architecture

Problem and objectives

A service can receive new requests faster than additional workers become ready. Keeping maximum capacity available avoids some overload but wastes allocated resources during quieter periods. A purely reactive controller also waits for demand to be observed before it requests capacity. Our objective is to predict near future demand, provision an appropriate number of workers and limit service degradation when the prediction is wrong.

The operating targets are a 95th percentile response latency at or below 250 ms and at least 99% successful requests. Normal experiments allow one to four worker replicas. The design chooses the smallest bounded allocation that covers the estimated demand, then applies readiness and stabilization rules. These targets guide the controller and evaluation; they are not an unconditional service guarantee.

Architecture and control loop

The host runs an open loop traffic generator and an ML controller. Real HTTP requests pass through a gateway pod to a Kubernetes Service and its ready worker endpoints. Workers execute a CPU bound PBKDF2 task and expose health, readiness and telemetry. The gateway starts a new upstream connection per request so traffic can be distributed across workers.

```
Load generator -> HTTP gateway -> Service -> Ready worker pods
Actual sends + latency + CPU + pod state -> ML controller
ML forecast + safety policy -> Kubernetes Deployment replica count
```

Every ten seconds, the controller forms a completed demand window, predicts the next 30 second peak, reads desired and ready capacity, and selects a new replica target. A desired replica is a requested allocation; a ready replica can serve traffic. Pending pods remain separate because requesting a pod does not make its capacity immediately available. The browser viewer displays the observed signals and the decision history.

Scope of the implementation

The deployment runs on Docker Desktop and Minikube on one laptop. It uses the Kubernetes API to change real application replicas. It does not provision AWS instances, Azure virtual machines or additional physical nodes. This platform gives a reproducible demonstration without requiring a paid cloud account. Because all replicas share one host, the findings describe application allocation within that host rather than cloud fleet capacity.

2 Implementation platform and input data

Table 1 Implementation environment

Component	Implemented configuration
Container platform	Docker Desktop Linux containers; Minikube acc-lab; Kubernetes v1.34.0; one node configured with 2 CPUs and 2300 MB memory.
Worker service	PBKDF2 HMAC SHA256 with 60000 iterations per request; one active CPU task and a bounded asynchronous queue.
Worker resources	CPU request and limit 250m; memory request 32 MiB and limit 96 MiB.
Availability handling	Five second readiness delay; ready endpoints receive requests; termination uses a pre stop delay and graceful drain.
Host software	Python 3.11 or later; NumPy for regression; Matplotlib for analysis; kubectl and Minikube for cluster access.
Recorded output	Arrival schedule, actual requests, decisions, resource samples, saved model and run summary.

Input data and demand definition

The generator follows a seeded Poisson arrival schedule with periodic, ramp and flash traffic profiles. These are controlled synthetic workloads, replayed as actual network requests. Each request record includes its send time, completion status and end to end latency. Demand is the actual number of requests sent per second within a completed ten second window. Using completed throughput as demand would hide some overload because failed or delayed work would reduce apparent demand.

Model features and control signals

Table 2 Input features and prediction target

Input or target	Definition
Six lag features	The six most recent completed demand windows, covering 60 seconds.
Four derived features	Mean, standard deviation and maximum of those six values; slope defined as latest minus oldest divided by five.
Prediction target	Maximum actual request rate in the next three ten second windows, covering 30 seconds.
Additional policy signals	p95 latency, failed request fraction, metric freshness, desired replicas, ready replicas and pending pods.

Only past demand enters the forecasting feature vector. Scenario names, future arrivals and known burst times are excluded. Future windows are used afterwards to score forecasts, not to make scaling decisions.

3 Machine learning methodology

Algorithm and training process

We use standardized Ridge regression. Lagged demand features are strongly correlated, so L2 regularization helps control coefficient magnitude while retaining fast inference and a compact model. A linear model is also easier to inspect in a short lab demonstration. The implementation solves the regression using NumPy; scikit learn documentation is a mathematical reference rather than a runtime dependency [4].

```
Ridge objective = sum of squared prediction errors + alpha * sum(w_j^2)
Forecast target at time t = max(demand at t+1, t+2, t+3)
```

Feature means and scales are fitted on training rows. The intercept is unpenalized. Training traces vary cycle period, amplitude, phase and noise. Twelve independent sequences with seeds 0 to 11 produce 8544 supervised training rows; seeds 50 to 55 produce 4272 validation rows. Windows do not cross sequence boundaries. Validation selects alpha from 1, 10 and 100; alpha 10 is selected. A separate seed 99 trace provides the model only holdout.

Demand is transformed to the training scale using 70 divided by the calibrated safe request rate, then predictions are transformed back into requests per second. The saved JSON model stores coefficients, feature means and scales, alpha, the calibrated rate and horizon. During a run, the saved model performs inference after six windows are available. The system uses offline training followed by online prediction; it does not continuously retrain.

Accuracy and baseline comparison

Table 3 Prediction accuracy on independent data

Evaluation setting	Ridge MAE	Persistence MAE
Independent synthetic holdout	1.67 RPS	2.61 RPS
64 aligned recorded replay targets	11.24 RPS	9.39 RPS

Mean absolute error is the average absolute difference between the predicted and actual next 30 second peak. On the synthetic holdout, Ridge RMSE is 2.07 RPS. Persistence predicts that the most recent observed rate will continue. On the recorded replay targets, a past only five window mean has MAE 12.06 RPS. Ridge improves over persistence on falling demand slices but does not win overall. The synthetic accuracy therefore does not establish superior forecasting during real execution.

The validation 90th percentile of positive underprediction residuals supplies an empirical capacity buffer. It is extra headroom, not a calibrated confidence interval or a guarantee that 90% of future workloads will be covered.

4 Calibration and resource allocation

Measured capacity and forecast horizon

Before scaling, we load tested a single worker at increasing offered request rates under the 250m CPU limit. A retained rate passes only when its repeats satisfy both run and nonempty window latency and success targets and the generator remains valid. We then reserve 20% headroom below the highest repeatably passing rate.

Table 4 Measured worker capacity

One worker offered load	Observed result
18 RPS	Repeated passing runs; p95 approximately 103 to 109 ms; no failures.
24 RPS	Near the latency boundary; a repeat contains violating windows even though its run p95 is below 250 ms.
30 RPS	p95 approximately 1502 ms; about 2.63% failed requests.
Selected safe capacity	0.8 x 18 = 14.4 RPS per worker.

A two worker check at 28.8 RPS and a four worker check at 57.6 RPS both completed with 100% successful scored requests. Their run p95 values were approximately 68 ms and 110 ms. Across sampled scale outs, window end to ready delay had a median of 8.9 seconds and p95 of 9.6 seconds. The 30 second horizon covers that observed delay, although an unexpected burst can still arrive before a useful forecast. This delay includes controller and proxy work and sampling granularity; it is not exact container boot time.

Allocation rule and cost objective

```
Observed need = ceil(observed RPS / 14.4)
Forecast need = ceil((forecast RPS + validation buffer) / 14.4)
Raw need = max(observed need, forecast need)
Target = clamp(raw need, minimum replicas, effective replica cap)
```

With the same positive cost per worker, the smallest integer allocation covering the estimated demand is also the cheapest allocation under this capacity model. The effective cap is the smaller of the configured maximum and the number affordable under the hourly rate budget. The normal range is one to four replicas. This is a bounded provisioning heuristic, not a global optimizer. Additional latency rescue and downscale stabilization can deliberately retain more capacity.

5 Safeguards and implemented functionality

The controller separates forecasting quality from service protection. A wrong or unavailable prediction must not be the sole reason for reducing capacity. The following behavior is implemented in the running controller and covered by recorded execution or automated checks.

Table 5 Controller functions and safeguards

Function or condition	Implemented behavior
Predictive scale out	Requests capacity for the buffered 30 second forecast; actual demand can request a larger allocation.
SLA rescue	If p95 exceeds 250 ms or failures exceed 1%, requests at least one additional replica when no scale out is pending, subject to the cap.
Pending capacity	Tracks desired, ready and pending pods separately; waits for an outstanding scale out before further latency rescue or downscale.
Stable downscale	Retains the maximum recommendation over 30 seconds and removes at most one replica per decision.
Invalid forecast	Inference exceptions, nonfinite values and negative output trigger observed demand fallback and prevent model failure downscale.
Missing or stale demand	Holds the current replica count when demand telemetry is stale or invalid.
Budget and hard limit	Bounds allocations and records CAPACITY_LIMITED when the demand estimate exceeds affordable or permitted capacity.
Shadow mode	Records proposed targets and sends test traffic without changing deployment replicas, including startup and cleanup.
Monitoring and audit	Displays live or completed demand, forecasts, latency, failures and replica traces; saves raw records and decision reasons.

A dedicated shadow check generated seven recommendations and zero scaling actions. Deployment replicas remained one and generation remained 81 before and after the run. This is separate evidence from the active comparison suite. Seventeen automated tests passed, including policy fallbacks, shadow behavior and network related checks.

A zero forecast is a valid numerical value and is retained for deliberate failure injection. It differs from an exception or invalid value. Section 9 shows whether independent demand and latency signals can protect the service under that specific fault.

6 Execution demonstration

Startup and first model prediction

The captured walkthrough replays a four minute flash workload with guarded scaling, seed 707 and a 60 second warmup. It is a separate demonstration run, not one of the twelve matched benchmark runs. The dashboard cards show the most recently observed state before the latest decision is applied.

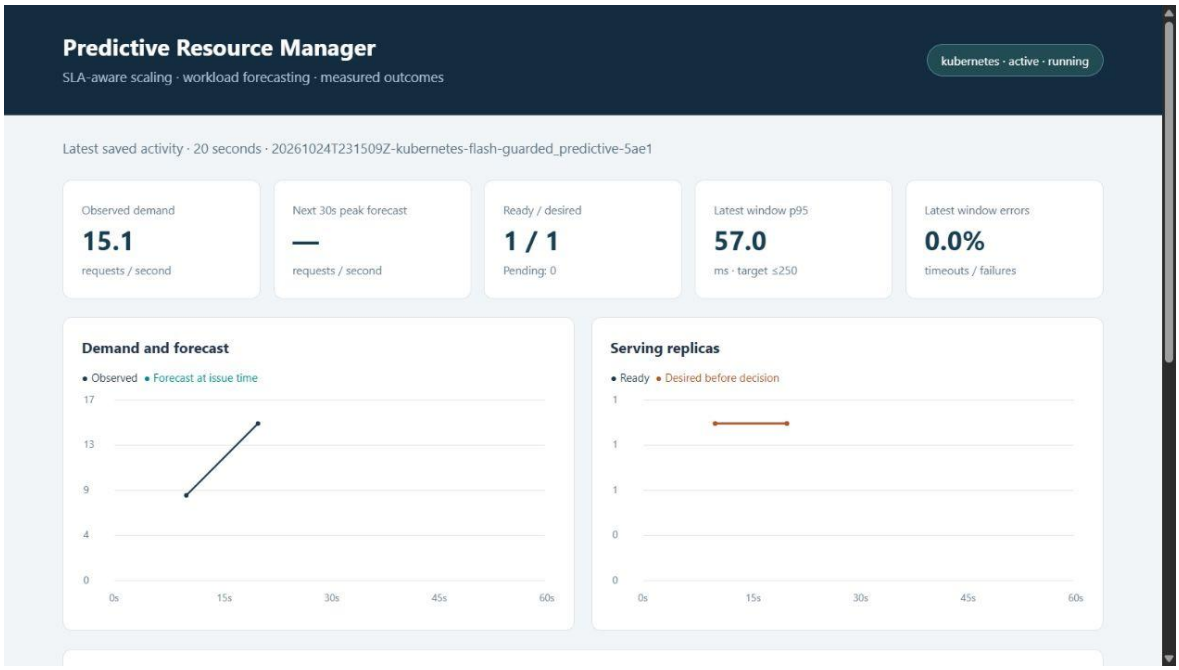


Figure 1 At 20 seconds the model is still warming up. Observed demand is 15.1 RPS, so the controller requests two workers even without an ML forecast.

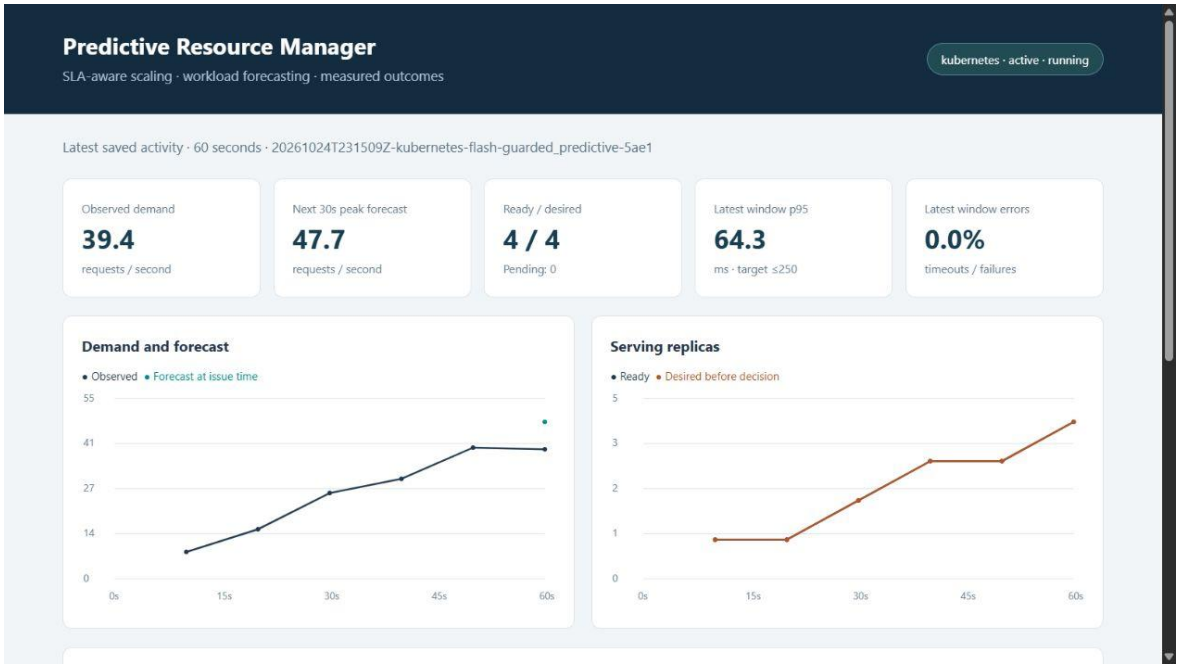


Figure 2 At 60 seconds six demand windows are available. Observed demand is 39.4 RPS, the model forecasts 47.7 RPS, four workers are ready and this window has p95 latency of 64.3 ms.

6 Execution demonstration continued

Downscale and workload recovery

The same run shows that the controller can reduce allocation after demand falls and respect the replica ceiling through a later burst. These views demonstrate state transitions rather than static interface features.

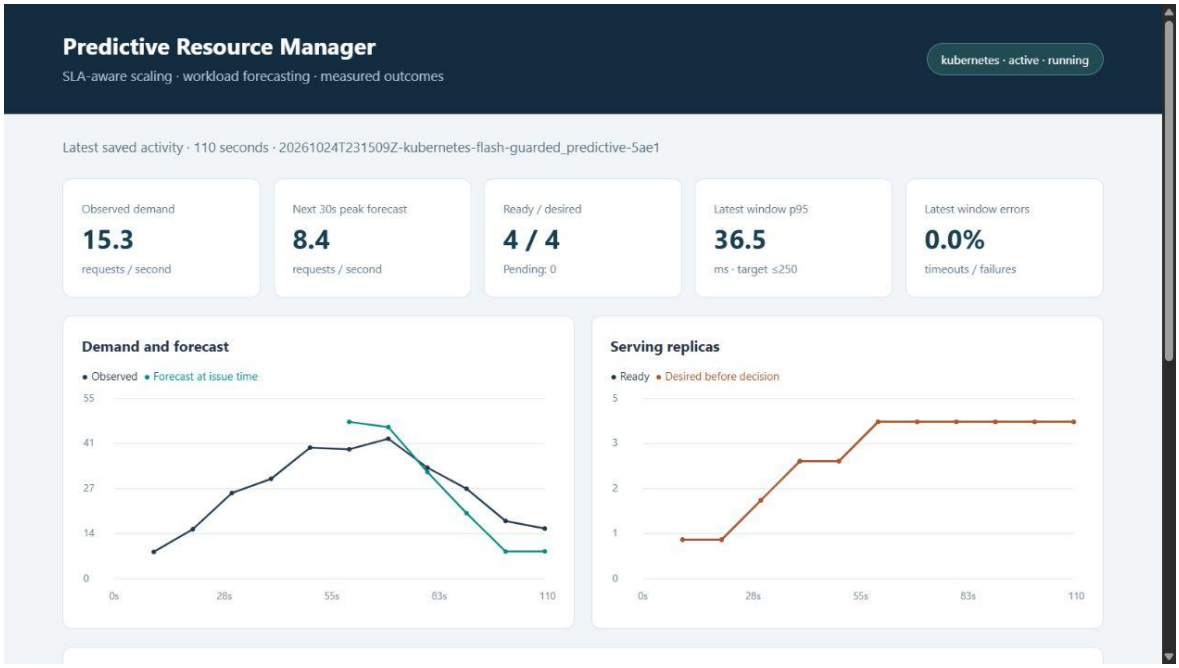


Figure 3 At 110 seconds demand is 15.3 RPS and the forecast is 8.4 RPS. Stabilization permits a target of three workers; the displayed ready and desired count is still four before that target is applied.

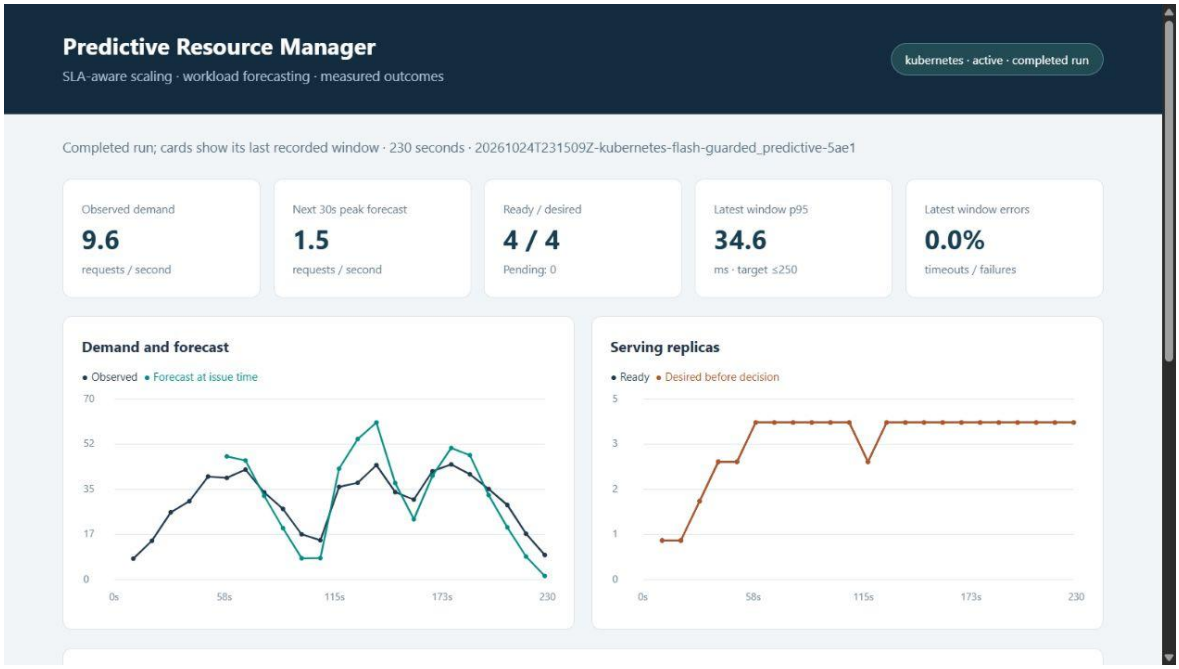


Figure 4 The completed trace records the flash burst and recovery. Actual allocation never exceeds four workers. Cards show the final recorded window; the complete walkthrough has 100% successful scored requests and a run p95 of about 49.7 ms.

7 Experimental method and performance results

Controlled comparison

The main suite contains twelve complete Kubernetes runs: three policies, two workloads and two independent seeds per workload. Every run lasts 240 seconds; the first 60 seconds are excluded from scored metrics. Policies use identical planned arrival schedules for each workload and seed, the same service code, frozen model, CPU limits, readiness delay and one to four replica envelope. Run order is randomized with a fixed seed. The audit confirms four matched schedules, identical source and model fingerprints, and zero generator capacity drops.

Fixed scaling keeps four workers. The custom reactive baseline scales from observed demand without using the forecast for its decisions. Guarded predictive scaling adds the forecast, validation buffer and service safeguards. The reactive baseline is custom code, not Kubernetes Horizontal Pod Autoscaler. The separate forecast only ablation is reserved for the failure demonstration.

Latency and service reliability

Table 6 Performance across matched runs

Workload and policy	Run p95 range ms	Mean replica seconds	Violating windows
flash / fixed	42.8 to 43.2	720.0	0.00%
flash / guarded	44.0 to 46.9	705.3	0.00%
flash / reactive	64.1 to 65.8	561.3	0.00%
periodic / fixed	34.6 to 55.3	720.0	0.00%
periodic / guarded	49.0 to 56.6	643.1	0.00%
periodic / reactive	71.5 to 503.9	498.8	5.56%

All twelve runs have 100% successful scored requests. A scored ten second window violates the operating target if its successful request p95 exceeds 250 ms, it has no successful requests, or more than 1% of its requests fail. The table reports the mean fraction of violating windows across two runs; p95 ranges are the observed minimum and maximum run p95, not confidence intervals or a pooled percentile.

On periodic traffic, guarded scaling uses 10.7% fewer replica seconds than fixed scaling and avoids the reactive baseline latency excursion. On flash traffic the saving versus fixed is only 2.0%. Reactive scaling uses fewer resources than guarded scaling in both workloads. This supports a resource versus service quality tradeoff, rather than a claim that predictive control is universally cheapest or fastest.

8 Utilization cost and energy

Definitions and measurement boundary

Worker CPU utilization is sampled process CPU time divided by allocated worker CPU limit time. With a 250m limit, one worker supplies 0.25 CPU cores. Replica seconds integrate desired worker allocation over the scored period. Gateway, control plane and host overhead are excluded consistently across policies. Higher utilization can indicate less idle headroom, but it can also increase queueing and tail latency.

```
Cost proxy = replica seconds / 3600 x 0.02 units per replica hour
Power proxy in W = 10 x allocated workers + 20 x used CPU cores
Energy proxy in Wh = integral of power proxy over seconds / 3600
```

The cost rate and power coefficients are explicit assumptions. CPU use and allocation are measured inputs to those formulas. They do not represent an actual provider bill, electricity meter reading or physical power saved by removing a pod. All workers run on the same laptop.

Table 7 Utilization and resource proxies

Workload and policy	CPU utilization	Cost units	Energy Wh
flash / fixed	27.2%	0.00400	2.272
flash / guarded	28.1%	0.00392	2.234
flash / reactive	35.0%	0.00312	1.831
periodic / fixed	22.2%	0.00400	2.222
periodic / guarded	25.4%	0.00357	2.013
periodic / reactive	31.9%	0.00277	1.607

Interpretation and sensitivity

For periodic traffic, guarded control increases utilization from 22.2% under fixed capacity to 25.4% and reduces estimated cost from 0.00400 to 0.00357 units. Its energy proxy decreases from 2.222 to 2.013 Wh. Reactive scaling has higher utilization and a smaller allocation, but the latency results show its service quality risk.

The assumed idle term drives much of the energy difference. If idle power were zero, the proxy would largely reflect CPU work, which is similar across policies because they serve the same workload. Increasing assumed idle power makes allocation reduction appear more beneficial. A physical energy claim would require host power measurement and a deployment that can release or power down resources.

9 Prediction failure and recovery

Deliberately incorrect forecasts

To test the question of wrong prediction directly, we multiplied model output by zero and replayed the same periodic arrival schedule with seed 404. Forecast only control can remove needed capacity; guarded control can still use observed demand and service signals. Each screenshot records one window, while its caption distinguishes the whole run outcome.

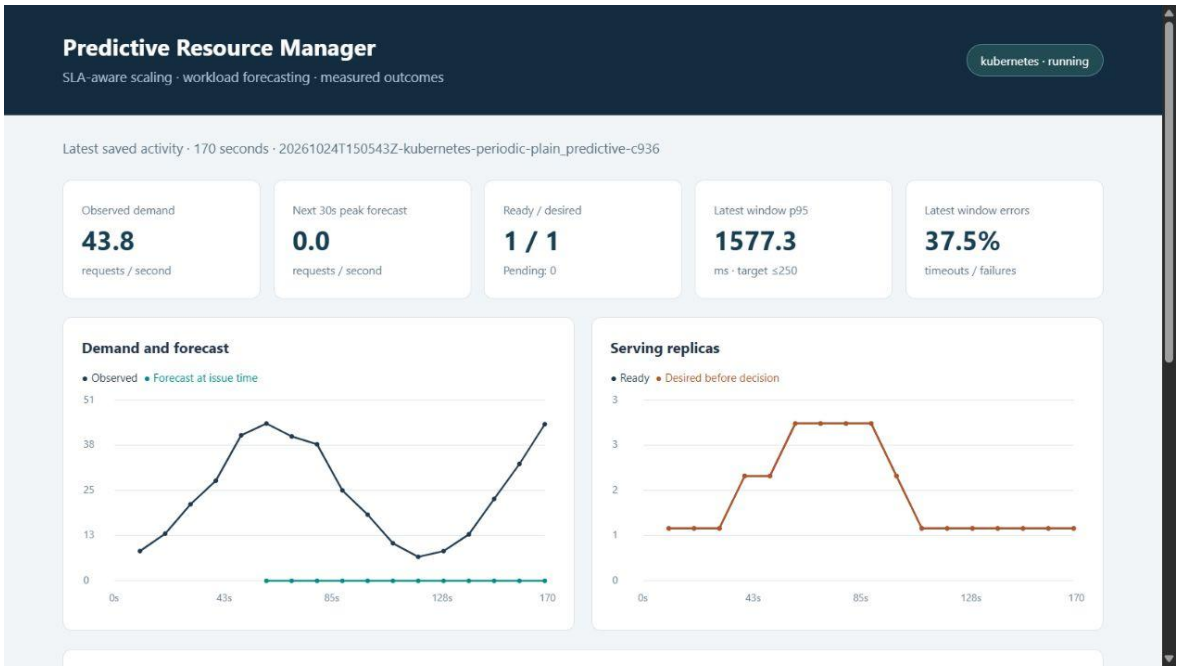


Figure 5 Forecast only control under the zero output injection. The displayed window has p95 1577.3 ms and 37.5% failures. Across the scored run, 13.35% of requests fail and p95 is 1547.7 ms.

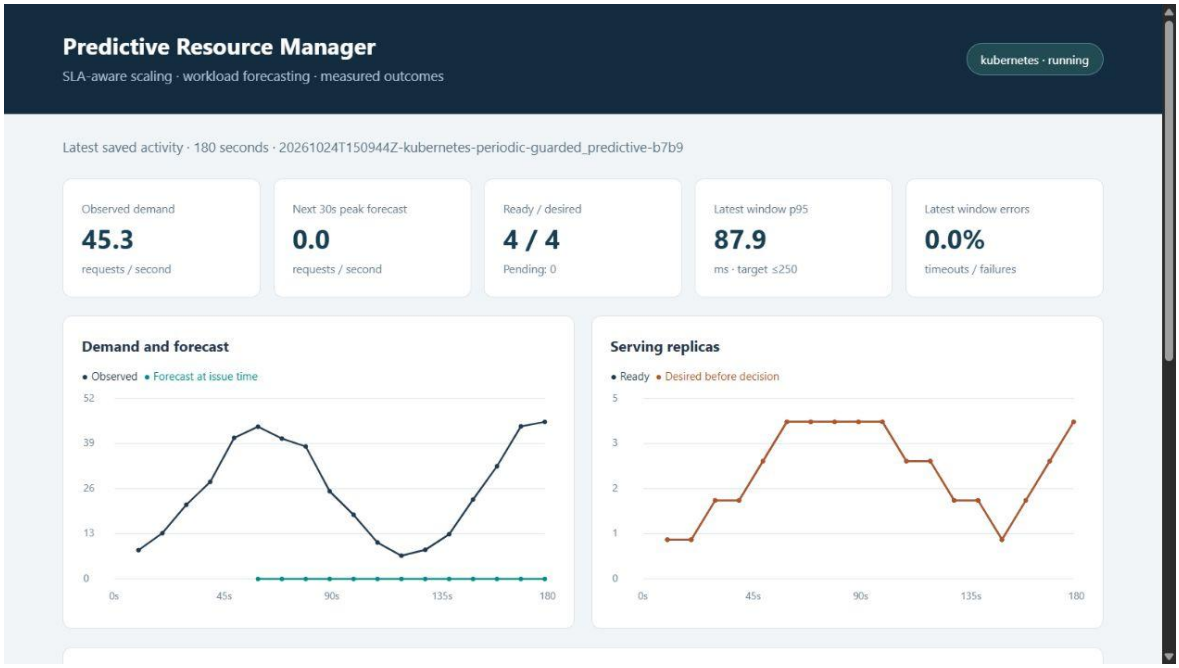


Figure 6 Guarded control under the same injection restores ready capacity despite the zero forecast. The displayed window has p95 87.9 ms and zero failures. The scored run has zero failed requests and p95 215.1 ms; one scored window still violates the target.

9 Prediction failure and constraints continued

Failure outcomes and the budget conflict

Table 8 Outcomes under injected prediction failure and a hard budget

Test and policy	Run p95 ms	Failed requests	Violating windows
Zero forecast / forecast only	1547.7	13.35%	38.89%
Zero forecast / guarded	215.1	0.00%	5.56%
One worker budget / guarded	1563.2	17.73%	66.67%

The guarded failure run still has one violating window out of eighteen scored windows. Rescue requires observation and readiness time, so it cannot erase all temporary harm. This paired injection tests the mechanism, not how often a naturally trained model fails. There is one run per policy for this demonstration, separate from the main matched suite.

The budget stress run uses a different seed, 505, a 120 second duration and an hourly budget of 0.02 units at 0.02 units per replica hour. This permits only one worker. The controller respects the budget, but 17.73% of scored requests fail and p95 reaches 1563.2 ms. It is a separate feasibility test, not a matched comparison with the zero forecast pair. A forecast cannot create capacity that a hard limit forbids.

What different prediction errors cause

Underprediction delays provisioning. The observed demand requirement and SLA rescue can request workers independently of the model, but readiness delay still exposes the service to queueing or failures. An unseen burst with no precursor is particularly difficult to predict from recent demand alone.

Overprediction reserves more workers than needed. Stabilization limits oscillation but intentionally delays resource release, so repeated overprediction increases replica time and the cost proxy. The hard cap bounds the maximum allocation. Invalid inference falls back to observed demand and prevents model failure downscale; stale or invalid demand holds the current state until telemetry is usable again.

10 Limitations and conclusion

Study limitations

The implementation executes real network requests and Kubernetes scaling, but the workload is synthetic and all replicas share one laptop. Two seeds per workload provide a controlled lab comparison, not a reliable estimate of performance across production traffic. Host contention, service routing and sampling granularity influence the measured timing. Per worker calibration must be repeated after hardware, CPU limit or workload changes.

The original predictor performs worse than persistence on the combined recorded replay targets. A separate exploratory model trained on recorded demand is retained in the repository, but it was not substituted into the frozen benchmark. Better model selection needs more independent traces and fresh matched controller runs. The current system does not implement automatic drift triggered retraining, multi node provisioning, cloud billing integration or measured physical energy optimization.

Service quality gains belong to the combined guarded controller. The main suite does not isolate the forecast from every other guard, so the results do not prove that Ridge alone causes the improvement. The zero forecast ablation demonstrates the value of observed demand and latency protection. More repeated experiments and an observed demand controller with otherwise identical safeguards would strengthen causal attribution.

Conclusion

The project meets its implementation objective by connecting a trained demand predictor to real resource allocation and measured service behavior. It identifies ten past only features, forecasts a 30 second peak and turns that estimate into bounded Kubernetes replica requests. The walkthrough demonstrates warmup, model inference, scale out, stable downscale, burst handling and the hard replica ceiling.

The controlled evaluation covers prediction accuracy, utilization, latency, cost and energy. Guarded scaling preserves the operating targets throughout the main suite while reducing allocation versus fixed maximum capacity, with a 10.7% reduction on periodic traffic. Its extra headroom costs more than the reactive baseline. Incorrect forecast and budget tests show both the benefit and limits of service safeguards. The result is a working resource manager with transparent tradeoffs, rather than a claim of perfect prediction or guaranteed SLA compliance.

11 Reproduction and references

Source code and execution

Repository: https://github.com/parth1056/accml
Implementation branch: parth/predictive-scaler

The repository contains the application, Kubernetes manifests, training code, raw experiments, analysis and the three person presentation. The report is based on retained execution records from 24 and 25 October 2026. Repository access must be available to the evaluator.

From the repository root, with Python, Docker Desktop, Minikube and kubectl available, the following commands reproduce the workflow. Calibration is host dependent; substitute its safe rate if the current machine reports a different value.

```
python -m pip install -r requirements.txt
.\SetupLab.ps1
python -m scaler.experiment calibrate --backend kubernetes --rates 6 12 18 24 30 --
seconds 40
python -m scaler.forecast --safe-rps 14.4
.\RunDemo.ps1
python -m scaler.viewer
```

The viewer opens at http://127.0.0.1:8787. Run it in another terminal to inspect live and completed results. RunDemo.ps1 reads the saved calibration. Shadow mode uses .\RunDemo.ps1 -Mode shadow against an existing deployment. The README includes the complete comparison and fault test commands.

Evidence is retained under output/live and output/evaluation. Each completed run includes requests.csv, decisions.csv, resources.csv, model.json and summary.json. The frozen matrix is output/live/matrix.json; its audit, matched results and forecast diagnostics are under output/evaluation. Demo screenshots are under output/evidence. These records allow captions and reported results to be checked against execution.

References

[1] Advanced Cloud Computing Lab CA assignment and submission requirements supplied for the July to November 2026 assessment.

[2] Kubernetes documentation. Deployments. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ (accessed 25 October 2026).

[3] Minikube documentation. Docker driver. https://minikube.sigs.k8s.io/docs/drivers/docker/ (accessed 25 October 2026).

[4] scikit learn documentation. Linear models and Ridge regression. https://scikit-learn.org/stable/modules/linear_model.html (accessed 25 October 2026).

[5] Project source and execution evidence. https://github.com/parth1056/accml/tree/parth/predictive-scaler.